



TIP101 | Intro to Technical Interview Prep

Intro to Technical Interview Prep Summer 2026 (@ Section 1 | Tuesdays and Thursdays 3PM - 5PM PT)
Personal Member ID#: **151287**

Need help? Post on our [class slack channel](#) or email us at support@codepath.org

[Getting Started](#)[Overview](#)[Session #1](#)[Session #2](#)[Assignment](#)[Cheatsheet](#)[Learning with AI ✨](#)[Resources](#)[Learning with AI](#)[▶ Course Info](#)[Course Progress](#)[Unit 1](#)[Unit 2](#)[Unit 3](#)[Unit 4](#)[Unit 5](#)[Unit 6](#)[Unit 7](#)[Unit 8](#)[Report](#)

Unit 2 Cheatsheet

Here's a quick reference sheet for Unit 2. While not an exhaustive list, it highlights the key syntax and concepts you'll use in this unit, plus a few optional ideas that may help with problem-solving. You're still expected to know required material from earlier units.

Sections are labeled for clarity:

- In-Scope: May appear on the assessment
- Not In-Scope: Useful, but not required



Unit Goals

- ☐ Create and modify a dictionary
- ☐ Access and loop over keys, values, and key-value pairs in a dictionary
- ☐ Solve problems using frequency maps
- ☐ Solve problems involving lists, dictionaries, or a combination of a lists and dictionaries

▼ General Concepts In-Scope

Built-In Functions

Type Casting

Type casting is the process of transforming or 'casting' data from one type into another type.

`int(x)` Cast a float or string to an integer

- Accepts one parameter `x`: the float or string to turn into an integer
 - Floats will be rounded down
- Returns `x` as an integer.

Example Usage:

```
x = int(2.5) # x will be 2
y = int("5") # y will be 5
```

`float(x)` Cast an integer or string to a floating point number

- Accepts one parameter `x`: the integer or string to turn into a float
- Returns `x` as a float.

Example Usage:

```
x = float(2) # x will be 2.0
y = float("5") # y will be 5.0
z = float("5.3") # z will be 5.3
```

`str(x)` Transform a data type into a string

- Accepts one parameter `x`: a data type, commonly an integer, to turn into a string
 - `x` can also be more complex data types like lists or dictionaries!
- Returns `x` as a string.

Example Usage:

```
x = str(2) # x will be "2"
y = str(2.0) # y will be "2.0"
z = str([1, 2, 3, 4]) # z will be "[1, 2, 3, 4]"
```

Remainder Division

The `%` **operator** is also known as the modulo operator and performs remainder division. `x % y` returns the remainder after dividing `x` by `y`. [Try it](#)

Example Usage:

```
print(5 % 2) # Prints 1 because 5 / 2 = 2 remainder 1

print(10 % 2) # Prints 0 because 10 / 2 = 5 remainder 0
```

Sum

`sum(x)` Returns the sum of all values in a list. [Try it](#)

```
sum([1, 2, 3, 4]) # Evaluates to 10
```

Min

`min(x)` Returns the argument with the lowest value or the item with the lowest value in a list. [Try it](#)

```
# Example 1: Minimum item in a list
min([2, 3, 4, 1, 5]) # Evaluates to 1

# Example 2: Argument with minimum value
min(5, 2, 3) # Evaluates to 2

# Example 3: Smallest string
min(['a', 'b', 'c', 'aa']) # Evaluates to 'a'

# Example 4: Smallest key in a dictionary
min({'a': 1, 'b': 2, 'c': 3, 'd': 4}) # Evaluates to 'a'
```

Max

`max(x)` Returns the argument with the highest value or the item with the highest value in a list. [Try it](#)

```
# Example 1: Maximum item in a list
max([2, 3, 5, 1, 4]) # Evaluates to 5

# Example 2: Argument with maximum value
max(5, 2, 3) # Evaluates to 5

# Example 3: Maximum string
max(['a', 'b', 'c', 'aa']) # Evaluates to 'c'

# Example 4: Largest key in a dictionary
max({'a': 1, 'b': 2, 'c': 3, 'd': 4}) # Evaluates to 'd'
```

Dictionary Methods & Syntax

Setting and Updating Values

You can add new key-value pairs or update the value of an existing key using the assignment operator `=`. If the key does not exist, a new key-value pair is added to the dictionary.

```
d['d'] = 4          # Adds a new key 'd' with value 4
print(d)           # Outputs: {'a': 1, 'b': 2, 'c': 3, 'd': 4}

d['a'] = 100        # Updates the value of key 'a'
print(d)           # Outputs: {'a': 100, 'b': 2, 'c': 3, 'd': 4}
```

Accessing Elements

Dictionaries in Python allow you to access values using the keys. This can be done primarily in two ways:

Method 1: Direct access by Key

You can access the value associated with a specific key directly using square brackets `[]`. If the key is not found, this method will raise a `KeyError`.

```
d = {'a': 1, 'b': 2, 'c': 3}
print(d['a']) # Outputs: 1
print(d['b']) # Outputs: 2
```

Method 2: Using the `get()` Method. [Try it](#)

Alternatively, you can use the `get()` method, which provides a safer way to access values. The `get()` method returns `None` if the key is not found, or a default value that you can specify.

`d.get(key, default_val)` Returns the value of the item in the dictionary `d` with the specified key.

- Accepts two parameters:
 - `key`: the key of the value you want to access. This is a *required* parameter
 - `default_val`: default value to return if the specified key does not exist. This is an *optional* parameter. If no default value is specified, will return `None` for non-existent keys.
- Returns the value of item paired with `key`.

Example Usage:

```
d = {'a': 1, 'b': 2, 'c': 3}
print(d.get('a'))      # Outputs: 1
print(d.get('z'))      # Outputs: None
print(d.get('z', 'Not Found')) # Outputs: 'Not Found'
```

Pop Method

`d.pop(key, default_val)` [Try it](#)

- Accepts two parameters:
 - `key`: the key of the item you want to remove. This is a *required* parameter.
 - `default_val`: default value to return if the specified key does not exist. This is an *optional* parameter. If no default value is specified, will raise an error if no item with specified `key` exists.
- Returns the value of the removed item.

```
d = {'a': 1, 'b': 2, 'c': 3, 'd': 4}

# Example 1: Pop without default_val
d.pop('a') # Returns 1
print(d) # Prints {'b': 2, 'c': 3, 'd': 4}
d.pop('e') # Raises KeyError
```

```
d = {'a': 1, 'b': 2, 'c': 3, 'd': 4}

# Example 2: Get with default_val
d.pop('a', None) # Returns 1
print(d) # Prints {'b': 2, 'c': 3, 'd': 4}
```

```
d.pop('e', None) # Returns None  
print(d) # Prints {'b': 2, 'c': 3, 'd': 4}
```

Keys Method

`d.keys()` Returns a list of the keys in the dictionary. [Try it](#)

- Does not have any required parameters
- Returns a list of keys in the specified dictionary.

```
d = {'a': 1, 'b': 2, 'c': 3, 'd': 4}  
  
keys = d.keys()  
print(keys) # Prints ['a', 'b', 'c', 'd']
```

Values Method

`d.values()` Returns a list of the values in the dictionary. [Try it](#)

- Does not have any required parameters
- Returns a list of values in the specified dictionary.

```
d = {'a': 1, 'b': 2, 'c': 3, 'd': 4}  
  
values = d.values()  
print(values) # Prints [1, 2, 3, 4]
```

Items Method

`d.items()` Returns a list of the key-value pairs in a dictionary. [Try it](#)

- Does not have any required parameters
- Returns a list of key-value pairs in the specified dictionary. Each key-value pair is represented as a [tuple](#).

```
d = {'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

```
items = d.items()
print(items) # Prints [('a', 1), ('b', 2), ('c', 3), ('d', 4)]
```

[Close Section](#)

▼ Bonus Concepts Not In-Scope

The following syntax is nice to know and may improve your code readability or help you solve certain problems more easily. However, they are not *required* to solve any of the problems in this unit. These are **not in scope for the Unit 2 assessment**, and you do not need to memorize them! Click on the function to read more about how to use it.

- [remove\(x\)](#) Removes first element with value `x` from list.
- [zip\(\)](#) Allows iteration over multiple lists, dictionaries, or other [iterables](#) simultaneously.
- [defaultdict\(\)](#) Creates a dictionary with default values for new or missing keys. Helps to avoid `KeyError`s!
- [d.copy\(\)](#) Returns a copy of the dictionary `d`.
- [sorted\(x\)](#) Returns a copy of the given list, dictionary or other [iterable](#) sorted in ascending order.
 - [What's the difference between sorted\(\) and sort\(\)?](#)

[Close Section](#)